

گزارش استفاده از هوش مصنوعی در توسعه پروژه

این سند نحوه استفاده از هوش مصنوعی (Claude Code) در طول توسعه پروژه استریم موزیک مشابه اسپاتیفای را توضیح می‌دهد. پروژه در دو فاز توسعه داده شده است: فاز اول شامل پیاده‌سازی کامل فرانت‌اند (React + Vite + TypeScript) با داده‌های mock، و فاز دوم شامل پیاده‌سازی بک‌اند (Django REST Framework) و اتصال آن به فرانت‌اند.

۱. نقش هوش مصنوعی در توسعه

در طول این پروژه، هوش مصنوعی بیشتر به عنوان یک **دستیار فکری و مشاور معماری** استفاده شده است، نه صرفاً یک ابزار تولید خودکار کد. مهمترین نقش‌های آن به شرح زیر بوده‌اند:

الف) ایده‌پردازی و طراحی معماری Scalable

پیش از پیاده‌سازی هر بخش (چه در فرانت و چه در بک‌اند)، از هوش مصنوعی برای گرفتن ایده در مورد بهترین شکل پیاده‌سازی استفاده شد؛ به‌طوری‌که ساختار نهایی قابل توسعه (scalable) و قابل نگهداری باشد. برای مثال:

- در فرانت‌اند، تصمیم به جداسازی لایه‌ی `mock services` از UI (به شکلی که در فاز دوم فقط لازم باشد این لایه با API واقعی جایگزین شود، بدون تغییر در کامپوننت‌ها) با مشورت هوش مصنوعی گرفته شد. همین موضوع باعث شد مسیر انتقال از داده‌های mock در `/src/lib/mock` به سرویس‌های واقعی در `/src/lib/api` بسیار روان و کم‌هزینه انجام شود.
- در بک‌اند، تفکیک اپلیکیشن‌های Django بر اساس دامنه (، `notifications`، `payment`، `music`، `user` management) و جدا نگه‌داشتن لایه‌ی `services.py` از `views.py` (منطق تجاری در برابر لایه‌ی HTTP) با کمک هوش مصنوعی طراحی شد تا کد در آینده راحت‌تر تست و توسعه داده شود.

ب) کمک در فرآیند Debug

در هر دو فاز، هنگام برخورد با باگ‌ها یا رفتارهای غیرمنتظره (مثلاً مشکلات مربوط به رفرش شدن توکن در `client.ts`، یا خطاهای مربوط به پاسخ‌های نامنظم سرویس موزیک)، از هوش مصنوعی برای ردیابی علت اصلی مشکل و پیشنهاد راه‌حل استفاده شد.

ج) نوشتن تست‌ها

بخش قابل توجهی از تست‌های واحد (unit tests) در هر دو فاز — چه تست‌های Vitest در فرانت‌اند و چه تست‌های Django/pytest در بک‌اند — با کمک هوش مصنوعی نوشته شده‌اند، به‌خصوص برای پوشش سناریوهای موفق (happy path) و حالت‌های خطا (edge cases).

به‌طور خلاصه، هوش مصنوعی در نقش «همکار مشاور» عمل کرده: پیشنهاد ساختار، بررسی تصمیمات معماری، کمک به یافتن باگ، و نوشتن تست. تصمیم نهایی و پیاده‌سازی بخش عمده‌ی منطق کسب‌وکار توسط توسعه‌دهنده انجام شده است، به‌جز مواردی که در بخش‌های بعدی به آن اشاره می‌شود.

۲. نمونه کدهای پیاده‌سازی‌شده در هر فاز

فاز اول (Frontend)

در فاز اول، بیشترین کمک مستقیم هوش مصنوعی در نوشتن **فایل‌های تست** بوده است. دو نمونه از این فایل‌ها:

نمونه ۱: `frontend/src/lib/api/authService.test.ts`

این فایل رفتار تابع `logout` را در سه سناریو تست می‌کند: خروج موفق و پاک‌شدن `session` محلی، ادامه‌ی پاک‌سازی `session` حتی زمانی که فراخوانی `API` با خطا مواجه شود، و رد شدن از فراخوانی `API` وقتی `refresh token` وجود ندارد. این تست به‌طور کامل توسط هوش مصنوعی نوشته شده و از `vi.mock` برای شبیه‌سازی کلاینت `HTTP` استفاده می‌کند.

```
describe('authService logout', () => {
  it('still clears the local session when the logout API fails', async () => {
    {
      vi.mocked(client.post).mockRejectedValue(new Error('Network
unavailable'))

      await expect(logout()).resolves.toBeUndefined()

      expect(localStorage.getItem(ACCESS_TOKEN_KEY)).toBeNull()
      expect(localStorage.getItem(REFRESH_TOKEN_KEY)).toBeNull()
      expect(storage.get('current_user')).toBeNull()
    }
  })
})
```

نمونه ۲: `frontend/src/pages/authSchemas.test.ts`

این فایل اسکیمای اعتبارسنجی (`validation schema`) فرم‌های ثبت‌نام شنونده و هنرمند (`Zod schemas`) را تست می‌کند؛ از جمله بررسی تطابق پسورد، الزام پذیرش قوانین حریم خصوصی، و تابع `parsePortfolioLinks` که لینک‌های پرتفولیو را از رشته‌ی ورودی جدا می‌کند. این تست هم به‌طور کامل توسط هوش مصنوعی نوشته شده است.

```
it('parses comma and newline separated portfolio links', () => {
  expect(parsePortfolioLinks('https://one.test,
https://two.test\nhttps://three.test')).toEqual([
    'https://one.test',
    'https://two.test',
    'https://three.test',
  ])
})
```

فاز دوم (Backend)

در فاز دوم، بیشترین سهم مستقیم هوش مصنوعی در پیاده‌سازی ماژول پرداخت (`/backend/payment`) بوده است. اکثر منطق این ماژول — شامل ارتباط با درگاه پرداخت زرین‌پال، مدیریت خطاهای درگاه، محاسبه‌ی نرخ تبدیل ارز، و کنترلر (View) شروع و تایید پرداخت — توسط هوش مصنوعی پیاده‌سازی شده است.

نمونه: `backend/payment/services.py`

این فایل شامل تابع‌های `request_payment` و `verify_payment` است که با API زرین‌پال ارتباط برقرار می‌کنند، همراه با یک دیکشنری کامل از کدهای خطای زرین‌پال و پیام‌های متناظر فارسی/انگلیسی برای نمایش به کاربر:

```
def request_payment(*, amount, description, callback_url, metadata):
    if not settings.ZARINPAL_MERCHANT_ID:
        raise ZarinpalGatewayError(
            "Payments are not configured yet. Please contact support.",
            http_status=503,
        )

    return _post_to_zarinpal(
        "/pg/v4/payment/request.json",
        {
            "merchant_id": settings.ZARINPAL_MERCHANT_ID,
            "amount": get_gateway_amount(amount),
            "currency": "IRR",
            "description": description,
            "callback_url": callback_url,
            "metadata": metadata,
        },
    )
```

و `backend/payment/views.py` که فرآیند کامل شروع پرداخت (`SubscriptionPaymentCreateView`) و بازگشت از درگاه با تایید تراکنش (`SubscriptionPaymentCallbackView`) را به‌صورت اتمیک (با

`transaction.atomic` و `select_for_update` مدیریت می‌کند تا از `race condition` در فعال‌سازی اشتراک جلوگیری شود. علاوه بر منطق اصلی، تست‌های `backend/payment/tests.py` که سناریوهای موفق، خطای درگاه، و کنسل‌شدن پرداخت را پوشش می‌دهند نیز توسط هوش مصنوعی نوشته شده‌اند.

۳. نقاط قوت و ضعف هوش مصنوعی در هر فاز

فاز اول — فرانت‌اند

نقاط قوت:

- سرعت بالا در نوشتن تست‌های تکراری با الگوی مشابه (مثلاً تست‌های سرویس‌های `mock` که ساختار مشابهی دارند).
- پیشنهادهای خوب برای جداسازی لایه‌ها (`UI / state / service`) که مسیر مهاجرت به فاز دوم را ساده کرد.
- کمک مؤثر در دیباگ مشکلات مربوط به `state management (Zustand)` و رفتارهای غیرمنتظره در رندر کامپوننت‌ها.

نقاط ضعف:

- در برخی موارد نیاز به بازبینی دستی تست‌های تولیدشده بود، چون بعضی سناریوهای خاص `UI` (مثلاً رفتار `RTL` برای زبان فارسی) به‌خوبی توسط هوش مصنوعی درک نمی‌شد و نیاز به اصلاح داشت.
- در طراحی اولیه‌ی کامپوننت‌های پیچیده‌ی `UI` (مثل پلیر موزیک با کنترل‌های صف پخش)، پیشنهادهای هوش مصنوعی گاهی نیاز به ساده‌سازی داشتند چون انتزاع بیش از حد پیشنهاد می‌داد.

فاز دوم — بک‌اند

نقاط قوت:

- در پیاده‌سازی ماژول پرداخت، هوش مصنوعی در مدیریت دقیق خطاهای درگاه زرین‌پال (که تعداد زیادی کد خطا دارد) بسیار مؤثر بود و به‌صورت منظم و کامل آن‌ها را پوشش داد.
- رعایت نکات امنیتی مهم مانند استفاده از `transaction.atomic` و `select_for_update` برای جلوگیری از پردازش دوباره‌ی یک پرداخت (`double-processing`)، و بررسی مبلغ `verify` شده در برابر مبلغ اصلی درخواست.
- کمک قابل توجه در نوشتن تست‌های `APITestCase` با `mock` کردن دقیق درخواست‌های خارجی (زرین‌پال) بدون نیاز به تماس واقعی با سرویس.

نقاط ضعف:

- برخی جزئیات خاص دامنه (مانند نرخ تبدیل دلار به ریال `USD_TO_IRR_RATE` که باید به‌روز نگه داشته شود، یا حداقل مبلغ مجاز درگاه) نیاز به تایید و به‌روزرسانی دستی داشتند چون هوش مصنوعی اطلاعات لحظه‌ای از نرخ ارز یا قوانین

درگاه ندارد.

- در تعامل با API خارجی (زرین‌پال)، مستندات و رفتار واقعی درگاه گاهی با فرض‌های اولیه‌ی هوش مصنوعی متفاوت بود و نیاز به تست دستی در محیط sandbox زرین‌پال برای اطمینان از صحت پیاده‌سازی وجود داشت.
 - مدل‌سازی داده (schema) اولیه گاهی نیاز به بازبینی داشت تا با محدودیت‌های واقعی دیتابیس و نیازهای گزارش‌گیری تطابق کامل پیدا کند.
-

جمع‌بندی نهایی

هوش مصنوعی در این پروژه نقش یک دستیار توسعه در کنار برنامه‌نویس ایفا کرده است؛ عمدتاً برای گرفتن ایده در طراحی معماری قابل توسعه، کمک در فرآیند دیباگ، و نوشتن تست‌های واحد در هر دو فاز فرانت و بک‌اند. در بخش پرداخت بک‌اند، سهم هوش مصنوعی در پیاده‌سازی مستقیم کد بسیار بیشتر بوده است، در حالی که در فرانت‌اند بیشترین کمک مستقیم در نوشتن فایل‌های تست بوده و منطق اصلی برنامه توسط توسعه‌دهنده نوشته شده است.